

Plan de mejoras del Panel Administrativo

Análisis de deficiencias y guía de ejecución para corregirlas de forma segura, ordenada y sin romper datos en producción. Cubre diseño/UX, lógica, seguridad y mantenibilidad.

Veredicto de seguridad: sólido ✓

El render sanitiza con `escapeHTML()` de forma consistente, la escritura descansa en RLS de Supabase (no en la UI) y la clave del cliente es la *publishable*. **No hay XSS ni credenciales filtradas**. Los problemas reales son de correctitud / pérdida de datos y de consistencia de marca y accesibilidad.

Proyecto	Alcance	Rama de trabajo
Javy Suplementos	admin.html · login · dashboard JS/CSS · db.js · auth.js	claude → main (vía PR)
Total de hallazgos		
15 + pulido		

1. Mapa de los hallazgos

15 puntos priorizados por impacto. La columna clave es **dónde vive el arreglo**: lo que es **solo frontend** es seguro y desplegable ya; lo que toca **base de datos** requiere aplicar SQL en Supabase y decisiones de producto.

#	DEFICIENCIA	SEVERIDAD	VIVE EN
1	"Guardar inicio" no es atómico → puede borrar destacados/inicio	Crítico	Base de datos
2	"Pausar/Activar" reescribe la imagen con una ruta local	Crítico	Frontend
3	El estado "Guardando..." borra el icono del botón (permanente)	Crítico	Frontend
4	Drawer de producto sin semántica de diálogo ni foco	Alto	Frontend
5	El CTA primario no usa el verde de marca	Alto	Frontend
6	Subida de imágenes sin validar tipo ni tamaño	Alto	Frontend
7	Imagen huérfana en storage si falla el guardado	Alto	Frontend
8	Dos fuentes de verdad para "inicio" (formulario vs. sección)	Medio	Base de datos
9	Barra de guardado y toasts tapados por la tab-bar en mobile	Medio	Frontend
10	Errores crudos de Postgres mostrados al usuario	Medio	Frontend
11	Doble-submit en variantes/sabores	Medio	Frontend
12	Fuga de memoria con <code>createObjectURL</code>	Medio	Frontend
13	Refs sin <code>?.</code> pueden romper el init	Medio	Frontend
14	<code>css/admin.css</code> es CSS muerto y contradictorio	Medio	Frontend
15	Tablet (~768px) cae al modo mobile innecesariamente	Medio	Frontend

2. Hallazgos críticos

Corrupción de datos o feedback roto. Son la prioridad real.

CRÍTICO #1 "Guardar inicio" puede borrar datos (operación no atómica)

js/db.js:514-533

Primero hace un UPDATE masivo a **todas** las filas poniendo

`show_on_home=false, home_order=null, is_featured=false, featured=false`, y luego, en un bucle, pone `true` a los seleccionados (una query por producto).

- **No es atómico:** si una query del bucle falla tras el reset (red, RLS, timeout), quedan **todos los destacados e inicio borrados** y solo algunos restaurados, sin rollback.
- **Mezcla conceptos:** apaga `featured/is_featured` de todo el catálogo, aunque "destacado" es un flag independiente con su propio checkbox.

Cómo arreglarlo: mover la operación a una función RPC/transacción en Postgres (un solo

`update ... set show_on_home = (id = any($1))` con `home_order` vía `array_position`) y dejar de tocar `featured/is_featured` aquí (desacoplar "inicio" de "destacado").

CRÍTICO #2 Un simple "Pausar/Activar" reescribe la imagen con una ruta local

js/admin-dashboard.js:888-902 + js/db.js:196-200

`normalizeProductFromDb` sustituye `image_url` por una imagen local de `product-data.js`. `toggleProductAvailability` hace `{ ...product }` con ese objeto normalizado y lo persiste → reescribe la imagen (y recalcula slug, `precio_centavos`, etc.) en Supabase. Lo mismo al precargar el formulario.

Cómo arreglarlo: en el admin trabajar con el registro **crudo** de la BD; para el toggle, enviar un update **parcial**

`{ is_available, available }` en vez de reescribir todo el producto. (Solo frontend — no requiere migración.)

CRÍTICO #3 El estado "Guardando..." se come el icono del botón (permanente)

js/admin-dashboard.js:270-281

`setButtonLoading` guarda/restaura con `button.textContent`, pero el botón contiene `<span class="btn-icon">...SVG...</span>`. Tras el primer guardado, el icono del disquete desaparece para siempre mientras el drawer siga montado.

Cómo arreglarlo: usar un `<span>` de texto dedicado dentro del botón + una clase `is-loading` (igual que el login con `.login-spinner`), sin tocar el `textContent` del botón completo.

3. Hallazgos de severidad alta

ALTO #4 Drawer de producto sin semántica de diálogo ni manejo de foco

admin.html:266 · js/admin-dashboard.js:796, 1051

El formulario CRUD principal no tiene `role="dialog"`, `aria-modal` ni `aria-labelledby` (el panel de filtros **sí**). Escape no lo cierra, no hay focus-trap, y al cerrar no devuelve el foco al botón que lo abrió.

Cómo arreglarlo: replicar el patrón que **ya funciona** en el panel de filtros: semántica de diálogo, cierre con Escape, trampa de foco y restaurar foco al disparador. Reutilizar, no inventar.

ALTO #5 El CTA primario no usa el verde de marca

css/admin-dashboard.css:10 vs css/tokens.css:27

El admin define `--admin-green:#08e063` en paralelo en vez de usar `--cta:#00e676`. "Guardar"/"Agregar" no combinan con el sitio público. En el login el botón principal es **gris translúcido**: solo se pone verde en éxito.

Cómo arreglarlo: mapear las variables locales a los tokens (`--admin-green: var(--cta)` , etc.) y dar color de marca al CTA del login.

ALTO #6 Subida de imágenes sin validar tipo ni tamaño

js/db.js:539-558

No valida `file.type` ni `file.size`; la extensión sale del nombre (falsificable) y el bucket es público. Un admin puede subir 50 MB o un no-imagen sin feedback.

Cómo arreglarlo: validar `image/*` y un límite (~5 MB) antes de subir, mostrando el error en `setFormMessage`.

ALTO #7 Imagen huérfana en storage si falla el guardado

js/admin-dashboard.js:848-857

Sube la imagen primero; si `createProduct/updateProduct` falla después, queda basura en el bucket sin referencia.

Cómo arreglarlo: intentar `storage.remove()` del path si la mutación falla, o subir la imagen al final del flujo.

4. Hallazgos de severidad media

MEDIO #8 Dos fuentes de verdad para "inicio"

js/admin-dashboard.js:746-749 + js/db.js:340

El límite de 4–8 solo se valida en la sección Inicio; vía el formulario se puede marcar `show_on_home` sin límite y `getHomeProducts` corta a 8 arbitrariamente.

Cómo arreglarlo: unificar la gestión de inicio en un solo flujo / validar el conteo al guardar el formulario.

MEDIO #9 Barra de guardado y toasts tapados por la tab-bar en mobile

css/admin-dashboard.css:837-846, 1042-1050

En $\leq 860\text{px}$ el sidebar pasa a tab-bar fija inferior ($\sim 64\text{px}$), que solapa la barra sticky de guardado y los toasts — justo el feedback que más importa al guardar.

Cómo arreglarlo: en $\leq 860\text{px}$ subir `bottom` por encima de la tab-bar ($\sim 78\text{--}84\text{px}$).

MEDIO #10 Errores crudos de Postgres mostrados al usuario

js/admin-dashboard.js:862, 882, 899, 1007

Se muestra `error.message` directo: constraints o RLS aparecen como texto técnico en inglés.

Cómo arreglarlo: extender el helper `getFriendlyAuthError` (ya existe en `auth.js`) a errores de BD/duplicados.

MEDIO #11 Doble-submit en variantes/sabores

js/admin-dashboard.js:904-966

`addVariant` / `saveFlavor` no deshabilitan su botón durante el `await`. El índice único frena el duplicado, pero genera errores visibles.

Cómo arreglarlo: deshabilitar el botón mientras la promesa está pendiente.

MEDIO #12 Fuga de memoria con createObjectURL

js/admin-dashboard.js:1078

Cada cambio de archivo crea un object URL que nunca se revoca.

Cómo arreglarlo: `revokeObjectURL()` antes de crear el siguiente y al cerrar el drawer.

MEDIO #13 Refs sin `?.` que pueden romper el init

js/admin-dashboard.js:1056, 1062-1065, 1181

Varios `els.*.addEventListener` sin guarda opcional; si falta un nodo, `init()` lanza y deja el panel a medio inicializar.

Cómo arreglarlo: uniformar con `?.`.

MEDIO #14 `css/admin.css` es CSS muerto y contradictorio

No está referenciado por ningún HTML y usa azul `#5ab4e9` como primario (contradice el verde actual). Riesgo de editar el archivo equivocado.

Cómo arreglarlo: eliminarlo (y su mención en CLAUDE.md).

MEDIO #15 Tablet (~768px) cae al modo mobile innecesariamente

`css/admin-dashboard.css:1114`

Colapsa a tab-bar + tarjetas ya en $\leq 860\text{px}$; en tablet vertical se desaprovecha el ancho.

Cómo arreglarlo: breakpoint intermedio que mantenga tabla/sidebar entre 768–1023px.

Pulido (severidad baja): variantes nuevas sin `aria-label` · campos obligatorios sin indicador visual · sin skeleton de carga ni distinción "0 productos" vs "sin resultados" · toggle de filtros a 42px (rompe el mínimo táctil de 44px) · paleta de error fuera de tokens · helpers duplicados (centralizar en `utils.js`) · `seedProductsFromLocalData` N+1.

5. Cómo trabajarlo correctamente

Reglas que manda este proyecto: sin tests ni CI (se verifica a mano con `/preview`), rama **claude** (nunca push a main), datos reales en Supabase, y algunos arreglos necesitan SQL que solo el dueño aplica.

Principio rector: separar lo *solo-frontend* (seguro, desplegable ya) de lo que *toca la base de datos* (requiere migración + decisión de producto). Y nunca mezclar refactor con bugfix en el mismo PR.

Lotes de trabajo (cada lote = 1 PR revisable)

Lote A · Quick wins seguros	solo frontend
Incluye: #3 icono de loading · #5 tokens de marca · #9 solapamiento mobile · #14 borrar CSS muerto · ajustes táctiles 44px. Por qué primero: cero backend, cambios visibles, bajo riesgo. Calienta el terreno. Verificar: <code>/preview</code> en mobile y desktop; guardar un producto y ver icono y toasts correctos.	
Lote B · Integridad de datos sin migración	solo frontend
Incluye: #2 toggle con update parcial · precarga del formulario con registro crudo · #13 guardas <code>?.</code> Por qué importa: resuelve el grueso del riesgo de corrupción de datos sin tocar Supabase. Verificar: pausar/activar un producto de prueba y confirmar en la tabla de Supabase que <code>image_url</code> no cambió.	
Lote C · Subida de imágenes	solo frontend
Incluye: #6 validar tipo/tamaño · #7 limpiar imagen huérfana · #12 <code>revokeObjectURL</code>. Verificar: subir un no-imagen y un archivo grande (deben rechazarse con mensaje) y forzar un fallo de guardado para confirmar que no queda basura en storage.	
Lote D · Accesibilidad del drawer	solo frontend
Incluye: #4 replicar en el drawer el patrón del panel de filtros (role=dialog, aria-modal, Escape, focus-trap, restaurar foco). Verificar: abrir y cerrar el drawer usando solo el teclado.	

Lote E • Integridad de "inicio"

base de datos

Incluye: #1 función RPC/transacción atómica · #8 una sola fuente de verdad · #10 errores amigables de BD.

Requiere decisión del dueño antes de codear:

- ¿"destacado" e "inicio" son flags independientes? (*recomendado: sí, desacoplar*)
- ¿El límite 4–8 es duro también desde el formulario?

Gate de despliegue: el PR **no se mergea a main** hasta que el dueño aplique la migración SQL en Supabase, o el RPC falla en producción.

Mitigación intermedia sin migración: dejar de tocar `featured/is_featured` en `updateHomeProducts` reduce el daño aunque no da atomicidad total.

Lote F • Deuda y pulido

solo frontend

Incluye (PR aparte): centralizar helpers en `utils.js` · indicador de campos obligatorios · skeleton / estado vacío diferenciado · paleta de error en tokens · breakpoint de tablet.

Por qué este orden (y no "severidad primero")

El instinto sería atacar #1 de una, pero es justo el que necesita decisión de producto + migración del dueño: arrancar ahí bloquea. Con **A** → **B** → **C** → **D** se entrega valor real y seguro de inmediato—incluido el grueso del riesgo de datos en B, que **no** depende de la BD—mientras se definen en paralelo las preguntas del Lote E.

Checklist antes de cada PR

1. Probado en `/preview` (mobile + desktop donde aplique).
2. Lotes con datos: verificado el estado en Supabase **antes y después**.
3. Sin errores en consola.
4. Commit enfocado, mensaje claro, en la rama **claude**.
5. PR **claude** → **main** para aprobación del dueño; los de BD, gateados a la migración.

Documento generado a partir de la auditoría de los agentes *diseñador* y *lógica* del proyecto, con verificación manual de los hallazgos críticos en el código fuente. · Javy Suplementos